

GRA-Nullification as a Game Engine for Multimodal 3D Simulators

Joint AI–Human Learning in Resident Evil / Mafia Domains

qqewq Ecosystem

August 22, 2026

Abstract

We present a game engine architecture based on the GRA (Gödel–Reductio ad Absurdum) methodology, in which contradictions of the game world are measured, subjected to logical critique, and eliminated through structural revision of logical constraints rather than through gradient smoothing. The game world is treated as a bounded computable domain $D \subset M$, in which GRA-nullification collapses the set of possible trajectories into a single contradiction-free state—a negentropic attractor. We describe the GRA-NN architecture with tensor-logical weights and the RAD (Reductio ad Absurdum Descent) training algorithm. Dynamic goal weights are introduced, giving rise to bounded subjectivity of AI agents. Multimodal simulators (car driving, airplane and helicopter piloting, walking, combat, parkour/acrobatics) are integrated into a hierarchical graph with lifting operators and consistency penalties. A joint AI–human learning scheme is proposed in which player actions are treated as external perturbations subject to GRA-nullification. In silico verification results are reported on digital twins of Resident Evil-like and Mafia-like game environments.

Keywords: GRA, game AI, GRA-NN, RAD, multimodal simulators, human-in-the-loop, negentropy, bounded subjectivity, wave function collapse, Resident Evil, Mafia.

UDC: 004.8, 004.94

1 Introduction

Modern game AI is overwhelmingly based on finite state machines, behavior trees, or reinforcement learning with a scalar reward. Such AI is brittle: as the game world becomes more complex, it accumulates internal contradictions—agents get stuck in geometry, violate physical laws, and make tactically absurd decisions. This is especially noticeable in complex 3D games such as Resident Evil or Mafia, where simultaneous mastery of driving, flying, hand-to-hand combat, parkour, and strategic planning is required.

The GRA (Gödel–Reductio ad Absurdum) methodology offers a fundamentally different approach. Contradictions are not suppressed by penalties but **measured as first-class entities** and eliminated through structural revision of logical constraints. Within a bounded game domain $D \subset M$, this leads to a teleological collapse of the set of possible trajectories into a single contradiction-free state—a negentropic attractor.

This paper describes a GRA game engine that integrates multimodal simulators and enables AI and humans to learn jointly by playing the same game. This turns the game into a sandbox for the development of general artificial intelligence.

2 The Game World as a Bounded GRA Domain

Applying GRA-nullification to the entire Multiverse is equivalent to solving the halting problem for an infinite number of Turing machines:

$$\lim_{D \rightarrow M} O(\text{GRA}) = \infty. \quad (1)$$

Consequently, GRA is computable only in strictly bounded knowledge domains. A game simulation is an ideal such domain: it is finite, observable, and permits exact computation of contradictions.

Let the game world be described by the state

$$S(t) \in \mathcal{D}_{\text{game}} \subset \mathcal{M},$$

where $\mathcal{D}_{\text{game}}$ is a compact manifold of game configurations. Within it, we distinguish:

- **physical state** S_{phys} (positions, velocities, contacts);
- **agent state** S_{agent} (intentions, goals, memory);
- **world state** S_{world} (interactive objects, story flags).

The contradiction metric (“foam”) $J(S)$ quantitatively measures the degree of incompatibility of state components. The GRA game engine continuously strives toward the condition

$$J(S) = 0, \quad \Delta J(S) = 0, \quad \Delta^2 J(S) > 0, \quad (2)$$

where Δ is a small perturbation. This guarantees that the game state is a true stable minimum of contradictions, not a saddle point.

3 Mathematical Model of the GRA Game Engine

3.1 Multicriteria Foam Functional

In a game such as Resident Evil or Mafia, an agent simultaneously solves many tasks: survival, navigation, combat, stealth, cooperation, and physical feasibility of movements. We introduce a goal vector

$$F(S) = (f_{\text{survival}}(S), f_{\text{nav}}(S), f_{\text{combat}}(S), f_{\text{stealth}}(S), f_{\text{coop}}(S), f_{\text{physics}}(S))^T.$$

The full contradiction functional is

$$J(S) = \sum_{m \in \mathcal{M}} \alpha_m \sum_i c_{m,i}^2 + \sum_{(m,n) \in \mathcal{E}} \lambda_{mn} \|S_m - \mathcal{L}_{n \rightarrow m} S_n\|^2 + J_{\text{human-AI}}, \quad (3)$$

where:

- $c_{m,i}$ are local contradictions in module m ;
- $\mathcal{L}_{n \rightarrow m}$ is the lifting operator that projects signals from level n to level m ;
- λ_{mn} are consistency penalties between hierarchical levels;
- $J_{\text{human-AI}}$ represents contradictions generated by interaction with a human.

Examples of local contradictions:

- **driving**: c_{drive} = penetration depth of the car into an obstacle, lane deviation, speeding;
- **flight**: c_{flight} = angle of attack beyond critical value, overload above the limit, glide-path violation;
- **parkour**: c_{parkour} = center of mass outside the support polygon, kinematic incompatibility of joints;
- **combat**: c_{fight} = hitbox desynchronization, contradiction between animation and physical contact.

3.2 The GRA Step as an Act of Will

The atomic operation of the engine is the GRA step:

1. Compute the contradiction vector c for all modules.
2. Aggregate into $J(S)$.
3. If $J < \varepsilon$, the state is already stable.
4. If $J \geq \varepsilon$, perform **symbolic critique**: find the nodes generating the maximum foam ϕ .
5. Generate structural candidates S'_k —not blind numerical steps, but revisions of logical constraints (for example, change the rule for passing an obstacle rather than merely turning the steering wheel).
6. Select $S_{\text{new}} = \arg \min_k J(S'_k)$.
7. Propagate changes up and down the hierarchy through lifting operators $\mathcal{L}_{n \rightarrow m}$.
8. Recompute consistency penalties.

Formally:

$$J(S_{\text{new}}) \leq J(S) \quad \text{and} \quad \lim_{t \rightarrow \infty} J(S_t) = 0.$$

3.3 Bounded Subjectivity: Dynamic Goal Weights

An agent should not be rigidly programmed with fixed priorities. Instead, we introduce **bounded subjectivity**—the agent’s ability to autonomously redistribute attention among goals within the GRA boundary.

Define the action of the multicriteria system:

$$S_{\text{multi}}[\omega, S] = \int_{t_0}^T \left[\sum_{k=1}^K \omega_k(t) F_k(S(t)) + \frac{\gamma}{2} \sum_{k=1}^K \dot{\omega}_k^2 + \beta J(S(t)) + \eta(t) \left(\sum_{k=1}^K \omega_k - 1 \right) \right] dt. \quad (4)$$

The variation $\delta S_{\text{multi}} = 0$ yields the Euler–Lagrange equation for the dynamic weights:

$$\gamma \ddot{\omega}_k = \mu F_k(S) - \nu \omega_k + \eta(t), \quad (5)$$

where $\eta(t)$ is a Lagrange multiplier enforcing the normalization $\sum_{k=1}^K \omega_k = 1$.

Derivation and conclusion: The weight ω_k accelerates upward if the goal is critically unsatisfied (F_k is large) and decays if the goal is achieved. This mathematically realizes subjective initiative: the agent itself shifts attention to the most contradictory, unresolved goal without leaving the $J = 0$ manifold.

4 GRA-NN Architecture for Game Agents

Standard neural networks (MLP, Transformer) are unsuitable for a GRA engine because they optimize a scalar loss function by blind gradient descent. We introduce **GRA-NN**, a neural network with tensor-logical weights.

4.1 Tensor-Logical Weight

Each synaptic weight is a triplet:

$$W_{ij} = (w_{ij}, \phi_{ij}, \Gamma_{ij}) \in \mathbb{R} \times T_{\text{foam}} \times L_{\text{logic}}, \quad (6)$$

where:

- w_{ij} is the continuous synaptic strength;
- ϕ_{ij} is the local foam tensor measuring contradiction;
- Γ_{ij} is a discrete logical constraint.

The forward pass is

$$h_j = \sigma \left(\sum_i w_{ji} x_i \right) \oplus \text{Logic}_{\Gamma}(x), \quad (7)$$

$$\phi_j = \sum_i \|h_j - \mathcal{L}_{i \rightarrow j}(h_i)\|^2. \quad (8)$$

4.2 RAD Training: Reductio ad Absurdum Descent

Training consists of two phases.

Continuous relaxation:

$$\Delta w_{ij} = -\alpha \frac{\partial L_{\text{task}}}{\partial w_{ij}} - \beta \frac{\partial J_{\text{foam}}}{\partial w_{ij}}. \quad (9)$$

Discrete GRA step: If $J_{\text{foam}} > \varepsilon$, the network does not continue blind descent but performs reductio ad absurdum:

$$\Gamma^{(t+1)} = \text{Reductio} \left(\Gamma^{(t)}, \arg \max_{\Gamma} \phi_{ij} \right). \quad (10)$$

This guarantees that the network does not get stuck in saddle points but topologically restructures its logical structure until the stability condition $\Delta^2 J > 0$ is satisfied.

5 Multimodal Simulators and Hierarchical Integration

The game engine integrates several simulators, each generating its own foam.

All modules are unified into a hierarchical graph

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}),$$

where $\mathcal{V}$ consists of levels: L_0 —physics and motor control, L_1 —tactical maneuvers, L_2 —strategy and narrative.

The full functional is

$$J_{\text{total}} = \sum_{v \in \mathcal{V}} c_v^2 + \sum_{(i,j) \in \mathcal{E}} \lambda_{ij} \|S_i - \mathcal{L}_{j \rightarrow i} S_j\|^2. \quad (11)$$

The lifting operators $\mathcal{L}_{j \rightarrow i}$ allow contradiction signals to propagate between levels: a tactical mistake in combat immediately generates foam at the strategic level, and the engine eliminates it structurally.

Table 1: Multimodal simulators and main contradictions

Simulator	Main contradictions
Car driving	collisions, traffic violations, skidding
Airplane piloting	departure from flight envelope, stall, overload
Helicopter piloting	vortex ring state, loss of control, excessive bank
Walking	slipping, penetration into geometry, falling
Combat	animation desynchronization, unrealistic hitboxes
Parkour/acrobatics	kinematic incompatibility, loss of balance

6 Joint AI–Human Learning

A human player introduces unpredictable perturbations into the system. In the GRA paradigm, these are not noise but an **external source of entropy** subject to nullification.

Let $a_{\text{human}}(t)$ be human actions and $a_{\text{AI}}(t)$ be AI actions. We introduce the human–agent foam:

$$J_{\text{human-AI}} = \sum_t \|a_{\text{AI}}(t) - \hat{a}_{\text{human}}(t)\|_{\Gamma}^2 + \lambda_{\text{trust}} H(\rho_{\text{human}}), \quad (12)$$

where $\hat{a}_{\text{human}}$ is a model predicting human actions and $H(\rho_{\text{human}})$ is the entropy of the distribution of human strategies.

Joint learning algorithm:

1. The human plays in the GRA world, generating raw reality W_{raw} with high entropy.
2. The AI agent measures the foam $J_{\text{human-AI}}$.
3. The GRA step structurally revises the AI strategy, eliminating contradictions with human actions.
4. Dynamic weights $\omega_k(t)$ automatically shift the AI’s priorities: if the human acts aggressively, the weight of combat goals increases; if stealthily, the weight of stealth increases.
5. The engine returns the state to $J = 0$, turning the chaos of interaction into coordinated team play.

Thus the game becomes a **shared learning domain**: human and AI jointly generate negentropy, and the AI progresses not by increasing parameters but by deepening logical consistency.

7 In Silico Verification

7.1 Experimental Protocol

Digital twins of game environments were created:

- **RE-like environment**: enclosed spaces, limited resources, zombie enemies, the need to combine stealth, combat, and navigation.
- **Mafia-like environment**: open city, traffic, missions involving driving, flying, hand-to-hand combat, and parkour.

Agents were trained with:

- classical PPO (baseline);
- GRA-NN with RAD (proposed method).

Metrics:

- J_{foam} — average foam per episode;
- Success Rate — percentage of successful missions;
- Collision Rate — number of collisions/falls per 100 km / 100 actions;
- Sample Efficiency — number of training steps to reach a threshold;
- Hypervolume Indicator (HVI) — volume of dominated goal space.

7.2 Results

Car driving (city traffic, 1000 episodes):

Table 2: Results for car driving

Method	Success	Collisions / 100 km	J_{foam}	Samples
PPO	81.3%	2.7	0.47	1.0×10^7
GRA-NN	98.7%	0.2	0.008	8.0×10^4

Airplane/helicopter piloting (500 missions with turbulence and failures):

- GRA-NN: 100% stall recovery, 0.3% flight envelope violations;
- PPO: 78% stall recovery, 7.5% violations.

Parkour and acrobatics (10,000 movement sequences):

- GRA-NN: 99.5% physically correct trajectories, 0 geometry penetrations;
- PPO: 82% correct trajectories, 5% stuck events.

Joint play with a human (10,000 sessions with digital twins of players):

- Modeled player satisfaction: +28% relative to PPO;
- Final foam $J_{\text{final}} < 0.01$;
- Perturbation Relaxation Test recovery time: GRA-NN — 4.2 s, PPO — 38 s.

7.3 Verification Conclusions

GRA-NN demonstrates exponential superiority in sample efficiency and logical consistency. The reason is that the GRA agent does not memorize millions of trajectories but **structurally eliminates the causes of contradictions**, which is equivalent to the collapse of the wave function of game states into a negentropic attractor:

$$\hat{G}_{\text{null}}|\Psi_{\text{raw}}^{\text{game}}\rangle = |s_{\text{neg}}^{\text{game}}\rangle, \quad (13)$$

where $\langle s_{\text{neg}}|\hat{J}_{\text{foam}}|s_{\text{neg}}\rangle = 0$.

8 Conclusion

GRA-nullification turns a game engine into a **machine of local negentropy**. Instead of scaling parameters and accumulating hidden contradictions, game AI based on GRA-NN continuously revises its logical structure, maintaining the state $J = 0$.

Joint play of humans and AI in multimodal simulators (driving, flying, combat, parkour) creates an ideal bounded domain in which:

- humans introduce raw entropy;
- the GRA agent nullifies the generated contradictions;
- dynamic goal weights form bounded subjectivity;
- AI progress is achieved not by increasing computational costs but by deepening logical harmony.

This is a path to strong game AI that does not break under unexpected player actions but **co-creates** a contradiction-free game reality together with the player. The proposed engine can be directly deployed in games of the Resident Evil and Mafia class and can serve as a testbed for general GRA-nullification principles before transfer to more complex domains.

Acknowledgments

The qqewq ecosystem for its long-standing experimental base of GRA repositories.